==
PC BUILD STORE - CONTRIBUTION DETAILS
==
Course: Object Oriented Programming
Project: PC Build Store Management System
Members: 4
==
MEMBER 1: ABDULHANAN
Email: rehanazizabdulhanan@gmail.com
==
GUI CONTRIBUTIONS:

- Designed and implemented the main Dashboard screen with navigation sidebar
- Built the Product Listing view with grid layout and filtering options
- Created the AddPartDialog for adding new components to inventory
- Developed the Reports panel with chart visualizations and export options
- Implemented the AI Chat interface with message bubbles and input field
- Designed the dark theme toggle button and theme settings panel
- Created the image caching UI feedback indicators and loading animations
- Built the streaming response display for real-time chat updates
Backend CONTRIBUTIONS:

- Implemented DashboardDAO for retrieving aggregated sales and inventory data
- Wrote ReportsDAO for generating monthly, quarterly, and yearly reports
- Developed the AIChatService for handling natural language queries
- Created ImageCacheService for efficient product image loading and storage
- Implemented streaming support in ChatController for real-time responses
- Wrote the ReportGenerator business logic for data aggregation
- Developed ProductSearchService with multi-criteria filtering
- Created UserSessionManager for handling login state across modules
Database/JDBC Work:

- Designed the Products table schema with proper constraints and indexes
- Wrote SQL queries for dashboard statistics and inventory summaries
- Implemented connection pooling configuration for better performance
- Created stored procedures for report generation queries
- Designed the ChatHistory table for AI conversation logging
- Wrote migration scripts for schema versioning
Testing & Debugging:

- Tested Dashboard load times and optimized database queries
- Fixed theme persistence issue across application restarts
- Debugged image caching memory leak in product thumbnails
- Validated AI chat response parsing for edge cases
- Tested report generation with large dataset scenarios
- Fixed dark theme color inconsistencies in dialog boxes
- Resolved streaming connection timeout issues in chat
Integration Contributions:

- Integrated AI Chat module with Dashboard navigation
- Connected Product Listing with AddPartDialog for seamless workflow
- Linked Reports module with database aggregation layer
- Unified dark theme application across all UI components
- Connected image caching with product display modules
- Coordinated with Haroon on Dashboard-Build Configurator integration
GitHub Commit Evidence:

- Dashboard.java: Main dashboard layout and components
- ReportsPanel.java: Report generation and display

- AIChatPanel.java: Chat interface with streaming
- AddPartDialog.java: Dialog for adding inventory parts
- ThemeManager.java: Dark/light theme management
- ImageCacheService.java: Image loading and caching
- DashboardDAO.java: Dashboard data access
- ReportsDAO.java: Report query implementations

==

MEMBER 2: AHMAD MUSHTAQ
Email: 2501803@students.au.edu.pk

==

GUI CONTRIBUTIONS:

- Built the complete Billing module UI with invoice generation
- Designed the Receipt View screen with detailed breakdowns
- Created the Print Preview and Print functionality interface
- Developed the Save Invoice dialog with file format options
- Built the Payment Processing panel with method selection
- Designed the Transaction History table view with sorting
- Created the Tax Calculator input and display components
- Implemented the Bill Summary panel with itemized totals

Backend CONTRIBUTIONS:

- Implemented BillingDAO for invoice CRUD operations
- Wrote TransactionDAO for payment and transaction records
- Developed BillingService for invoice calculation and validation
- Created TaxCalculator for regional tax computations
- Implemented PrintService for receipt generation and printing
- Wrote ExportService for saving invoices to PDF and text formats
- Developed PaymentProcessor for handling multiple payment methods
- Created InvoiceGenerator for formatted bill creation

Database/JDBC Work:

- Designed the Invoices table with foreign key relationships
- Wrote SQL queries for invoice retrieval and filtering by date
- Implemented Transactions table for payment logging
- Created indexes on InvoiceDate and CustomerID for performance
- Wrote queries for financial summary reports by period
- Designed the TaxRates table for configurable tax percentages

Testing & Debugging:

- Tested billing calculations with various tax scenarios
- Fixed rounding errors in currency display
- Debugged print preview layout issues on different printers
- Validated receipt formatting for different item counts
- Tested invoice save functionality to PDF format
- Fixed payment processing timeout during peak loads
- Resolved transaction history pagination bugs

Integration Contributions:

- Connected Billing module with Build Configurator for project costs
- Linked Transaction records with Dashboard reports
- Integrated Print Service with system printer drivers
- Coordinated with Aitzaz on Build Upgrade pricing
- Connected Invoice system with product inventory updates
- Unified billing display with Abdulhanan's dark theme

GitHub Commit Evidence:

- BillingModule.java: Main billing controller
- ReceiptView.java: Receipt display and formatting

- PrintService.java: Print and preview functionality
- InvoiceDAO.java: Invoice database operations
- TaxCalculator.java: Tax computation logic
- PaymentProcessor.java: Payment method handling
- TransactionDAO.java: Transaction data access
- BillSummary.java: Invoice summary calculations

==

MEMBER 3: AITZAZ
Email: 2501805@students.au.edu.pk

==

GUI CONTRIBUTIONS:

- Built the Build Upgrades module with upgrade path recommendations
- Designed the Compatibility Check panel with visual indicators
- Created the Component Comparison view for side-by-side analysis
- Developed the Upgrade History timeline display
- Built the Performance Impact preview showing benchmark estimates
- Designed the Cost Benefit analysis display for upgrades
- Created the Alternative Suggestions pop-up dialog
- Implemented the Compatibility Report viewer with status icons

Backend CONTRIBUTIONS:

- Implemented UpgradeDAO for storing and retrieving upgrade records
- Wrote CompatibilityService for checking component compatibility
- Developed UpgradeRecommendationEngine for suggesting upgrades
- Created PerformanceEstimator for benchmark predictions
- Implemented ComponentMatcher for finding compatible alternatives
- Wrote ValidationService for input validation across modules
- Developed AlertService for compatibility warning notifications
- Created HistoryTracker for upgrade audit trail

Database/JDBC Work:

- Designed the Upgrades table with version tracking
- Wrote SQL queries for upgrade history retrieval by build
- Implemented the CompatibilityRules table for validation logic
- Created queries for finding compatible parts by socket type
- Designed the Benchmarks table for performance estimates
- Wrote queries for component availability checking

Testing & Debugging:

- Tested compatibility checks across 50+ component combinations
- Fixed edge case where CPU and motherboard socket mismatch was missed
- Debugged upgrade recommendation algorithm for budget constraints
- Validated performance estimates against real-world benchmarks
- Tested compatibility report generation for complex builds
- Fixed upgrade history display for builds with 10+ upgrades
- Resolved alert notification stacking issue

Integration Contributions:

- Connected Compatibility Service with Build Configurator
- Linked Upgrade recommendations with Product Listing prices
- Integrated with Ahmad Mushtaq's Billing for upgrade costs
- Coordinated with Haroon on motherboard slot compatibility
- Connected upgrade history with Dashboard analytics
- Unified compatibility alerts with Abdulhanan's theme system

GitHub Commit Evidence:

- BuildUpgradesModule.java: Upgrade management UI
- CompatibilityCheckPanel.java: Compatibility validation display

- CompatibilityService.java: Core compatibility logic
- UpgradeDAO.java: Upgrade database operations
- UpgradeRecommendationEngine.java: Recommendation algorithm
- PerformanceEstimator.java: Benchmark predictions
- ComponentMatcher.java: Compatible part finder
- CompatibilityRules.java: Validation rule definitions

==

MEMBER 4: HAROON
Email: haroonikram2512@gmail.com

==

GUI CONTRIBUTIONS:

- Built the Build Configurator screen with drag-drop assembly
- Designed the Cart interface with quantity controls and totals
- Created the Category Tabs for browsing PC components
- Developed the Motherboard Slot view showing expansion options
- Built the Component Selection grid with filtering sidebar
- Designed the Build Summary panel with part list and pricing
- Created the Shopping Cart checkout flow interface
- Implemented the Motherboard Compatibility indicator display

Backend CONTRIBUTIONS:

- Implemented BuildDAO for storing custom PC configurations
- Wrote CartDAO for shopping cart persistence and management
- Developed BuildConfiguratorService for assembly validation
- Created CartService for cart operations and calculations
- Wrote MotherboardSlotService for slot availability checking
- Implemented CategoryService for component categorization
- Developed CheckoutService for order processing
- Created SearchService with category-based filtering

Database/JDBC Work:

- Designed the Builds table with component references
- Wrote SQL queries for build retrieval with joined components
- Implemented the CartItems table for shopping cart storage
- Created queries for category-based product filtering
- Designed the MotherboardSlots table for slot specifications
- Wrote queries for build cost aggregation

Testing & Debugging:

- Tested Build Configurator with various component combinations
- Fixed cart calculation errors with bulk discounts
- Debugged category tab switching performance issues
- Validated motherboard slot detection for 20+ models
- Tested cart persistence across application sessions
- Fixed checkout flow edge case with empty cart
- Resolved component selection state management bugs

Integration Contributions:

- Connected Build Configurator with Aitzaz's compatibility checks
- Linked Cart module with Ahmad Mushtaq's Billing system
- Coordinated with Abdulhanan on Dashboard build statistics
- Integrated Category Tabs with Product Listing filters
- Connected Motherboard Slot view with component validation
- Unified build assembly with product inventory updates

GitHub Commit Evidence:

- BuildConfigurator.java: Main build assembly screen
- CartInterface.java: Shopping cart management

- CategoryTabs.java: Component category navigation
- MotherboardSlotView.java: Slot display and management
- BuildDAO.java: Build configuration persistence
- CartDAO.java: Cart data access operations
- BuildConfiguratorService.java: Build validation logic
- MotherboardSlotService.java: Slot compatibility checking

==

CROSS-MODULE INTEGRATION

==

The four modules were integrated through:

1. Shared Database Layer: All DAOs use a common connection pool and schema
2. Event System: Modules communicate through observer pattern events
3. Theme Integration: Abdulhanan's dark theme applies to all modules
4. Navigation Hub: Dashboard serves as central navigation to all features
5. Data Flow: Build selections flow through compatibility checks,
   pricing calculations, and finally billing/invoice generation
6. Shared Models: Component, Build, and Cart models used across modules

Integration Order:

- Phase 1: Individual module development (Weeks 1-3)
- Phase 2: Module connection and data flow (Week 4)
- Phase 3: UI/UX unification and theme application (Week 5)
- Phase 4: Testing, debugging, and optimization (Week 6)

==

END OF FILE

==